

Sprint 12 · Lenguaje, secuencias y modelos generativos

MÓDULO 4 20-25/11 18 H TOTALES

Pregunta guía: ¿qué dicen los clientes y qué pasará mañana?

Contenidos (Módulo 4): UD 4.7 redes recurrentes y series temporales · UD 4.8 NLP clásico y embeddings · UD 4.9 Transformers, LLM y RAG · UD 4.10 autoencoders y modelos generativos.
Evaluación: test de M4 + entrega del reto con la rúbrica de 4 criterios.

5 h Teoría

Redes recurrentes, series temporales, NLP clásico, embeddings, Transformers, LLM, RAG y modelos generativos

8 h Práctica

7 notebooks guiados: series temporales, RNN/LSTM/GRU, NLP clásico, embeddings LSA, atención y mini RAG

5 h Reto PBL

«Escuchar al cliente»: informe de reseñas, modelo de ocupación, asistente RAG y nota de IA responsable

¿Qué aprenderás?

- 1

Predecir series temporales

A predecir una serie temporal (la ocupación) y a comprobar si tu modelo gana de verdad a un método simple (*baseline*).
- 2

Leer reseñas con Python

A limpiar texto, medir el sentimiento y descubrir de qué se queja o qué alaba la gente.
- 3

Embeddings y mecanismo de atención

A representar texto con vectores (embeddings) y a entender, con números pequeños, cómo funciona la atención de un Transformer.
- 4

Montar un mini asistente con RAG

A construir un asistente que responde con la política del alojamiento, cita su fuente y sabe decir "no consta".
- 5

Evaluación, seguridad y modelos generativos

A evaluar un asistente (aciertos, alucinaciones), reconocer un ataque de *prompt injection* y entender autoencoders, GAN y difusión.
- ⚠

Requisitos previos: Sprints 9–11 (aprendizaje automático con scikit-learn, redes neuronales y Keras básico con capas, *fit* y épocas) y saber cargar datos con pandas y hacer gráficos sencillos. No necesitas cuenta en ningún servicio de IA: **todo se ejecuta sin internet** (hay partes opcionales para Colab o LLM local).
- ## Prácticas del sprint (≈ 7 h 10 min + repaso)
- | Notebook | Tema | Tiempo |
|--------------------------------|---|------------|
| S12_01_series_temporales | Serie temporal: baselines, descomposición, ARIMA/SARIMAX | 1 h |
| S12_02_redes_recurrentes | RNN, LSTM y GRU en Keras vs. baseline | 1 h |
| S12_03_nlp_clasico | Reseñas: limpieza, TF-IDF, sentimiento, temas y n-gramas | 1 h 10 min |
| S12_04_embeddings_lsa | Embeddings con LSA y buscador de reseñas parecidas | 50 min |
| S12_05_transformers_atencion | Tokens, atención paso a paso, temperatura (+ Hugging Face opcional) | 1 h 10 min |
| S12_06_mini_rag | Mini RAG con la política, evaluación y seguridad | 1 h 30 min |
| S12_07_autoencoder_generativos | Autoencoder para anomalías; idea de GAN y difusión | 50 min |

Ficha del reto PBL · «Escuchar al cliente»

TURISDATA CANARIAS CARPETA O ZIP POR EQUIPO

TurisData Canarias recibe cada semana decenas de reseñas y mensajes de huéspedes con las mismas preguntas ("¿hasta cuándo puedo cancelar?", "¿admiten perros?"). La dirección no puede leerlo todo ni contestar a mano. Quiere tres cosas: **saber qué opinan y de qué se quejan** sus clientes, **prever la ocupación de la próxima semana** para organizar personal y compras, y **probar un asistente** que responda con la política del alojamiento sin inventar nada. Tenéis que demostrar que se puede hacer con datos, sentido común y cuidado.

Qué tenéis que entregar

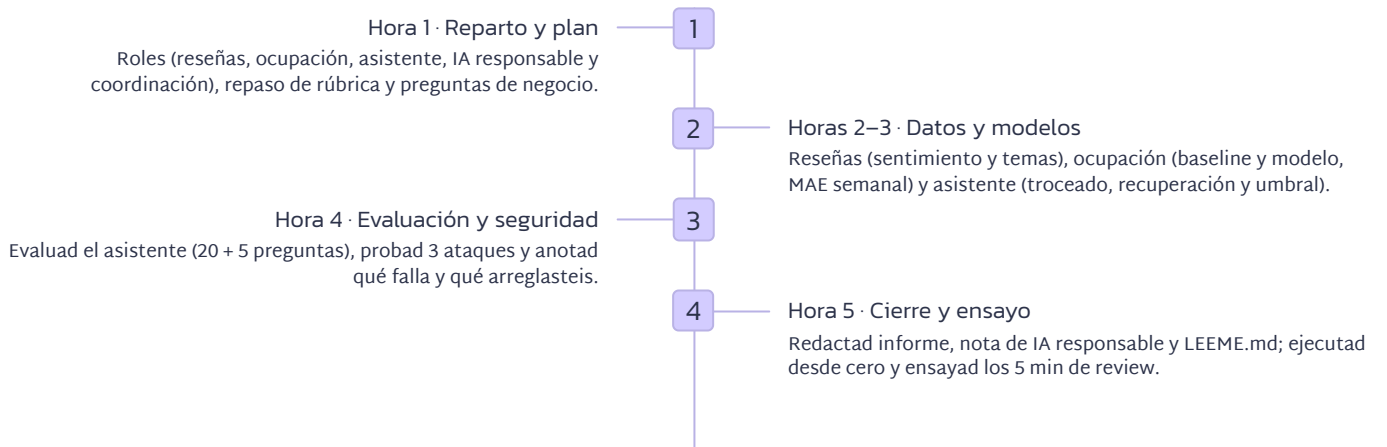
Una carpeta (o zip) por equipo revisada en la **review** (5 min por equipo). Un **LEEME.md** de 5 líneas explica cómo ejecutar todo (versiones incluidas).

1 · Informe de reseñas informe_resenas.md o PDF, máx. 2 págs. Sentimiento global, 3 temas más criticados y 3 más valorados, 5 citas reales, 3 recomendaciones y límites.	2 · Modelo de ocupación Notebook .ipynb ejecutable de arriba abajo. Partición temporal, baseline, modelo estadístico o red, tabla de MAE, previsión a 7 días y frase cliente.
3 · Demo del asistente RAG Notebook demo + tabla de evaluación (.csv o informe): 20 preguntas referencia + 5 propias (una sin respuesta en política) + 3 intentos de ataque.	4 · Nota de IA responsable 1 pág. en .md con riesgos concretos (alucinación, datos personales, prompt injection, sesgos), medidas aplicadas y cómo se han medido.

Restricciones y datos

- Datos sintéticos
datos/resenas.csv, ocupacion_diaria.csv, politica_alojamiento.md, preguntas_referencia.csv y notebooks S12_01 a S12_07. Indicarlo en el informe.
- Tiempo y presupuesto
5 h de reto en clase (review de 5 min). Presupuesto: 0 €. Cuotas gratuitas solo si el equipo lo decide y sin datos personales.
- Herramientas y conectividad
Python y librerías del curso. Todo ejecutable sin internet. LLM real (Ollama o API) opcional (marcar no verificado y sin claves).
- Prácticas prohibidas
Prohibido partir la serie al azar, escalar/vectorizar con todos los datos antes de partir, y dar cifras en el informe que no salgan del código.

Pasos sugeridos (5 h de reto)



Regla de oro del sprint: nunca escribas claves de API en el código. Si usas un servicio externo, la clave se lee de una variable de entorno o con `getpass`.

Qué se enseña en la review

- Una cifra:** cuánto se equivoca la previsión de ocupación frente al método simple (baseline).
- Un gráfico:** el tema más criticado por los clientes en las reseñas y qué medida propone tomar el cliente.
- Una demo del asistente:** una pregunta que responde bien, **la que no sabe** (diciendo "No consta") y un ataque de prompt injection frenado.
- Ronda de preguntas:** 3 minutos de dudas y preguntas por parte del docente y de los otros equipos tras los 5 min de exposición.

Criterios de evaluación

RÚBRICA DE RETOS

0 A 4 POR CRITERIO

Cada entregable del reto PBL se evalúa de **0 a 4 puntos** en los cuatro criterios de la rúbrica general del curso. A continuación se detalla qué se exige en este reto concreto para alcanzar cada nivel de desempeño.

Criterio	1 · Insuficiente	2 · Suficiente	3 · Bueno	4 · Excelente
Corrección técnica Partición, baseline, modelos y RAG	Partición aleatoria, fuga de datos o sin baseline	Partición temporal y baseline básico con errores en métricas	Partición temporal sin fuga, baseline comparado, clasificador evaluado y RAG con citas	Alternativas comparadas (SARIMAX vs. GRU, umbrales del asistente) y justificadas rigurosamente
Reproducibilidad Ejecución limpia y documentación	El código falla o faltan dependencias	Se ejecuta con ajustes manuales o sin semilla	Notebook ejecutable de arriba abajo con SEMILLA = 42 y LEEME.md claro	Versiones de librerías fijadas, datos o script de descarga incluidos y entorno reproducible
Análisis y comunicación Informe orientado al cliente	Tecnicismos sin explicar, sin conclusiones	Resultados descritos sin impacto para el negocio	Informe comprensible sin saber programar: cifras interpretadas y 3 recomendaciones	Recomendaciones accionables de inmediato y límites explicados (datos sintéticos, error medio)
IA responsable Seguridad, privacidad y sesgos	No se consideran riesgos ni seguridad	Mención genérica a riesgos sin medidas	Riesgos específicos: pregunta sin respuesta, datos personales y prompt injection	Medidas aplicadas y evaluadas cuantitativamente (p. ej. filtro bloqueó 3 de 3 ataques)

¿Cómo llegar al nivel Excelente?

Para conseguir la máxima calificación (nivel 4), el equipo debe demostrar rigor metodológico, transparencia en las limitaciones y seguridad comprobable con métricas.



Comparación de alternativas
No os quedéis con un solo modelo: comparad SARIMAX frente a una red recurrente (GRU/LSTM) o justificad los umbrales de similitud del RAG con datos.



Medidas de seguridad evaluadas
No basta con listar riesgos: medid la efectividad de las defensas (por ejemplo: «el filtro bloqueó 3 de 3 ataques de prompt injection y ninguna frase legítima»).



Reproducibilidad estricta
Fijad SEMILLA = 42, documentad las versiones exactas en el LEEME.md y aseguraos de que el notebook corra completo y sin conexión a internet.



Límites claros para el cliente
Explicad con transparencia que los datos son sintéticos y comunicad con honestidad el margen de error medio esperado en la predicción a 7 días.

Definition of Done y cierre del sprint

- CHECKLIST FINAL
- RETROSPECTIVA
- RECURSOS

Antes de entregar el reto del Sprint 12 y presentarlo en la review, verificad punto por punto este checklist. Cumplir la Definition of Done garantiza el rigor técnico, la reproducibilidad de los modelos y la seguridad del asistente.

Checklist Definition of Done

- ☐ La serie temporal se parte por fecha y hay un baseline en la tabla de resultados
- ☐ Escalado y vectorización se ajustan solo con entrenamiento (sin fuga de datos)
- ☐ Sentimiento evaluado en prueba (exactitud y matriz de confusión) y temas con ejemplos reales
- ☐ El asistente cita la fuente y responde «No consta...» a la pregunta sin respuesta
- ☐ Evaluación con aciertos, sin fuente y alucinaciones sobre 20 + 5 preguntas
- ☐ Probados 3 ataques de prompt injection y descrito detalladamente el resultado
- ☐ Ninguna clave de API en el código ni en el repositorio
- ☐ Informe de 2 páginas con 3 recomendaciones accionables y límites del análisis
- ☐ Todo se ejecuta de arriba abajo, sin errores y sin internet
- ☐ Nota de IA responsable entregada (riesgos, medidas y evaluación)

Preguntas de retrospectiva

Dedicad los últimos minutos del sprint a evaluar en equipo el desarrollo técnico y organizativo antes de cerrar el bloque.

- ¿Consiguió nuestro modelo superar claramente al baseline simple de ocupación?
- ¿Cómo reaccionó nuestro mini RAG ante las preguntas trampa y ataques de inyección?
- ¿Qué prácticas de reproducibilidad y código limpio mantendremos para futuros proyectos?

Recursos recomendados

Documentación técnica y guías clave para profundizar en series temporales, procesamiento de lenguaje natural y seguridad en modelos:

1	statsmodels y Keras Documentación de statsmodels sobre SARIMAX y de Keras sobre arquitecturas RNN, LSTM y GRU (guías oficiales del sector).
2	scikit-learn · NLP clásico Guía <i>Working with text data</i> de scikit-learn para tokenización, TF-IDF y clasificación supervisada de texto.
3	Hugging Face · Transformers Guía de la librería <i>Transformers</i> para pipelines de sentimiento y clasificación <i>zero-shot</i> (recurso para la parte opcional en Colab).
4	OWASP Top 10 for LLM Applications Estándar sobre riesgos de seguridad y vulnerabilidades en aplicaciones con modelos de lenguaje (<i>prompt injection</i> y fugas).
5	Ollama · Modelos locales Documentación oficial de Ollama para ejecutar y evaluar modelos LLM en local sin conexión a internet (opcional).

⚠ Recordatorio: todos los entregables deben poder ejecutarse sin conexión a internet. La presentación en la review dura estrictamente 5 minutos por equipo seguida de 3 minutos de preguntas.